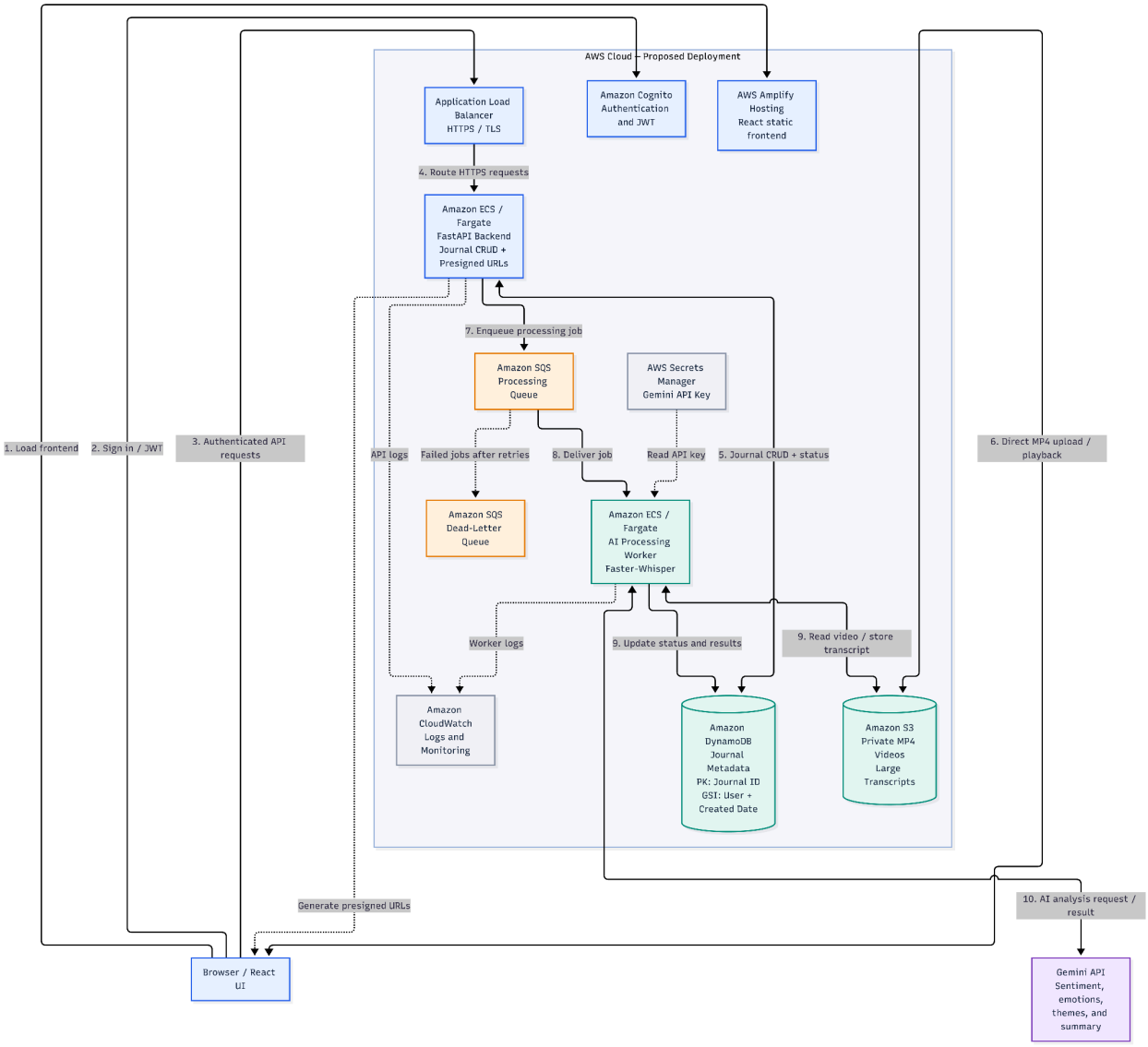


AWS Architecture Description — AI Video Journaling Platform

Names: Nyang Lin Phyo (nl2993), Aravindan Vemula (Av4008), ,Adhyayan Verma (av4159)

So, our proposed AWS architecture supports an AI video journaling application in which users upload private videos, receive speech transcripts and AI generated insights, and revisit past entries. The design separates the web interface, API, persistent data, and asynchronous processing so that each component can scale independently.

AWS Amplify Hosting serves the static React frontend, where users create journals, upload videos, and view results. Amazon Cognito handles registration and authentication, issuing

JSON Web Tokens (JWTs). The browser sends authenticated requests through an Application Load Balancer (ALB), which provides an HTTPS entry point and routes traffic to our FastAPI backend running on Amazon ECS with AWS Fargate. Fargate runs containers without requiring us to manage virtual machines. FastAPI handles journal creation, retrieval, deletion, processing requests, and authorization checks. It also validates JWTs to associate requests with authenticated users.

Amazon DynamoDB stores journal metadata, processing status, and compact AI results. Each record is identified by a journal ID, while a Global Secondary Index (GSI) on user ID and creation date supports displaying each user's journal history in chronological order. DynamoDB offers managed scaling and conditional writes, which help prevent invalid or duplicate status transitions. Its trade off is less flexible querying than PostgreSQL: additional search and analytics features would require deliberate index design. Because GSI reads are eventually consistent, newly created entries may briefly lag in journal history. Large transcripts are stored in Amazon S3 instead of DynamoDB to avoid its 400 KB item size limit.

Amazon S3 also stores private MP4 files. Rather than sending entire videos through FastAPI, the backend generates short lived presigned URLs so the browser can upload or play videos directly from S3. This reduces API bandwidth and compute load, but requires carefully scoped permissions, expiration times, upload validation, and coordination with the frontend. Once an upload is confirmed, FastAPI records the journal state and sends a processing message to Amazon Simple Queue Service (SQS).

SQS decouples video submissions from computationally expensive AI work. An ECS/Fargate worker consumes queued jobs, reads the MP4 from S3, and uses Faster Whisper to transcribe speech. It then calls the external Gemini API to generate sentiment, emotions, themes, and a summary. The worker saves results and updated status in DynamoDB, optionally storing a lengthy transcript in S3. The frontend retrieves progress and results by polling FastAPI. Because SQS can redeliver messages, workers must use idempotent updates to avoid duplicate processing. Messages that repeatedly fail are sent to an SQS dead letter queue for investigation.

AWS Secrets Manager protects the Gemini API key instead of embedding it in code, while Amazon CloudWatch collects API and worker logs and operational metrics. The main trade off is simplicity versus cost: Fargate and managed services reduce infrastructure maintenance, but long running transcription tasks can be expensive, and Faster Whisper may perform better on GPU backed ECS instances rather than CPU only Fargate. Using Gemini also introduces an external service dependency and data handling considerations. Overall, this design prioritizes modularity, private storage, and reliable asynchronous processing while leaving room for future optimization.